

1 P1

We have seen the importance of using gradient information to train neural networks efficiently.

There are essentially four ways in which the gradient of a neural network error function can be evaluated.

翻译：

我们已经看到了利用梯度信息来高效训练神经网络的重要性。

本质上，主要有四种方法可以用来评估神经网络误差函数的梯度。

2 P2

The first approach, which formed the mainstay of neural networks for many years, is to derive the backpropagation equations by hand and then to implement them explicitly in software.

If this is done carefully it results in efficient code that gives precise results that are accurate to numerical precision.

However, the process of deriving the equations as well as the process of coding them both take time and are prone to errors.

It also results in some redundancy in the code because the forward propagation equations are coded separately from the backpropagation equations.

As these often involve duplicated calculations, then if the model is altered, both the forward and backward implementations need to be changed in unison.

This effort can easily become a limitation on how quickly and effectively different architectures can be explored empirically.

翻译：

第一种方法多年来一直是神经网络的中流砥柱，即手动推导反向传播方程，然后在软件中显式地实现它们。

如果操作得当，这种方法能产生高效的代码，并给出精确到数值精度的准确结果。

然而，推导方程和编写代码的过程都非常耗时，且容易出错。

这还会导致代码出现一定的冗余，因为前向传播方程是与反向传播方程分开编写的。

由于这些过程通常涉及重复计算，因此如果模型发生更改，前向和后向的实现必须同步进行修改。

这种工作量很容易成为限制因素，影响我们快速有效地在经验上探索不同网络架构的能力。

重点与难点解读：

核心主旨：这段文字主要讨论了早期训练神经网络的主流方法——**手动推导并实现反向传播（Back-propagation）**。作者首先肯定了该方法在代码效率和数值精度上的优势，但随后重点剖析了其显著的缺陷：开发过程繁琐、易出错、代码冗余以及维护困难（前向与后向传播需同步修改）。这些缺点最终限制了研究人员快速尝试和验证不同网络架构的效率，从而引出了对更先进的自动微分技术的需求。

重点词汇与短语：

- **mainstay:** 中流砥柱，主要依靠。形容该方法在多年间的主导地位。
- **explicitly:** 显式地。指程序员需要手动、明确地写出每一个计算步骤。
- **numerical precision:** 数值精度。指计算机浮点数运算所能达到的精确程度。
- **prone to errors:** 容易出错。
- **redundancy:** 冗余。指代码中存在不必要的重复。

- **in unison:** 一致地，同步地。强调前向和后向代码修改时必须保持严格同步，否则会引发难以察觉的错误。
- **explored empirically:** 通过经验探索。指在实际应用中通过反复试验（试错）来寻找最优的网络结构。

难点解析：

- **理解“冗余（Redundancy）”的来源：**文中的冗余并非指代码写得不简洁，而是指由于前向传播（计算输出）和反向传播（计算梯度）被分开独立编码，导致相同的中间计算逻辑在两个地方重复出现。这不仅增加了代码量，也增加了维护成本。
 - **为何“容易出错（prone to errors）”：**神经网络的梯度推导涉及大量的链式法则运算，手动推导极其繁琐。更关键的是，代码实现时必须保证前向传播和反向传播的严格对应。一旦网络结构稍有改动，如果程序员忘记同步修改两处代码，就会导致梯度计算错误，而这种错误往往很难通过简单的代码审查发现，会导致模型无法收敛或收敛到错误的结果。
-

3 P3

A second approach is to evaluate the gradients numerically using finite differences.

This requires only a software implementation of the forward propagation equations.

One problem with numerical differentiation is that it has limited computational accuracy, although this is unlikely to be an issue for network training as we may be using stochastic gradient descent in which each evaluation is only a very noisy estimate of the local gradient.

The main drawback of this approach is that it scales poorly with the size of the network.

However, the technique is useful for debugging other approaches, because the gradients are evaluated using only the forward propagation code and so can be used to confirm the correctness of backpropagation or other code used to evaluate gradients.

翻译：

第二种方法是用有限差分法对梯度进行数值评估（即数值微分）。

这只需要对前向传播方程进行软件实现即可。

数值微分的一个问题是它的计算精度有限，尽管这对于网络训练来说不太可能成为一个问题，因为我们可能使用的是随机梯度下降，其中每次评估只是局部梯度的一个带有很大噪声的估计。

这种方法的主要缺点是它的可扩展性很差，难以适应大规模网络。

然而，该技术对于调试其他方法非常有用，因为梯度的评估只使用了前向传播代码，因此可以用来验证反向传播或其他用于评估梯度的代码的正确性。

重点与难点解读：

核心主旨：这段文字介绍了评估神经网络梯度的第二种方法——**数值微分 (Numerical Differentiation)**，并重点分析了其优缺点。其最大的优点是实现简单（只需前向传播）且能作为“标准答案”来调试复杂的反向传播代码；其致命缺点则是计算效率极低，无法适应大规模网络。

重点词汇与短语：

- **finite differences:** 有限差分。一种通过给输入增加微小扰动来近似计算导数的数学方法。
- **numerical differentiation:** 数值微分。利用有限差分来估算梯度的过程。
- **computational accuracy:** 计算精度。数值微分受限于步长选择和浮点数舍入误差，精度通常不如解析解（符号微分）。
- **stochastic gradient descent (SGD):** 随机梯度下降。深度学习中常用的优化算法，本身带有噪声，因此对梯度的精确度要求不那么苛刻。
- **scales poorly:** 可扩展性差。指随着问题规模（网络参数量）增大，计算成本急剧上升。
- **debugging:** 调试。此处指利用数值微分的结果来检验反向传播算法实现是否正确（即所谓的“梯度检查”）。

难点解析：

- 为何“计算精度有限”却不是大问题？文中提到，虽然数值微分本身精度不高，但在训练神经网络时，我们通常使用随机梯度下降（SGD）。SGD 每次只用一小批数据（mini-batch）来估计梯度，这个估计本身就带有极大的随机噪声。既然优化算法本身就在“噪声”中工作，那么数值微分带来的那一点点额外误差就显得微不足道了。
- 为何“可扩展性差（scales poorly）”？这是数值微分最致命的弱点。假设一个神经网络有 N 个参数（权重和偏置），解析方法（如反向传播）可以在与网络计算量相当的常数倍时间内算出所有

N 个梯度。而数值微分需要对每个参数单独进行一次扰动和前向传播计算，这意味着计算量会随着参数数量 N 线性增长。对于动辄拥有数百万甚至数十亿参数的现代深度网络，这种方法的计算成本是完全无法接受的。

4 P4 符号微分与表达式膨胀

A third approach is called *symbolic differentiation* and makes use of specialist software to automate the analytical manipulations that are done by hand in the first approach.

This process is an example of *computer algebra* or *symbolic computation* and involves the automatic application of the rules of calculus, such as the chain rule, in a completely mechanistic process.

The resulting expressions are then implemented in standard software.

An obvious advantage of this approach is that it avoids human error in the manual derivation of the backpropagation equations.

Moreover, the gradients are again calculated to machine precision, and the poor scaling seen with numerical differentiation is avoided.

The major downside of symbolic differentiation, however, is that the resulting expressions for derivatives can become exponentially longer than the original function, with correspondingly long evaluation times.

Consider a function $f(x)$ given by the product of $u(x)$ and $v(x)$.

The function and its derivative are given by

$$f(x) = u(x)v(x) \tag{8.42}$$

$$f'(x) = u'(x)v(x) + u(x)v'(x). \tag{8.43}$$

We see that there is redundant computation in that $u(x)$ and $v(x)$ must be evaluated both for the calculation of $f(x)$ and for $f'(x)$.

If the factors $u(x)$ and $v(x)$ themselves involve factors, then we end up with a nested duplication of expressions, which rapidly grow in complexity.

This problem is called *expression swell*.

翻译：

第三种方法被称为符号微分（symbolic differentiation），它利用专用软件将第一种方法中手工完成的解析推导过程自动化。

这一过程是计算机代数（computer algebra）或符号计算（symbolic computation）的一个实例，涉及在一个完全机械化的过程中自动应用微积分规则（如链式法则）。

随后，所得到的表达式会在标准软件中加以实现。

该方法的一个显著优点是避免了手动推导反向传播方程时可能出现的人为错误。

此外，梯度同样可以计算到机器精度，并且避免了数值微分中存在的扩展性差的问题。

然而，符号微分的主要缺点在于，其生成的导数表达式长度可能相对于原函数呈指数级增长，导致相应的计算时间大幅增加。

考虑一个由 $u(x)$ 与 $v(x)$ 相乘得到的函数 $f(x)$ 。

该函数及其导数分别为：

$$f(x) = u(x)v(x), \quad (8.42)$$

$$f'(x) = u'(x)v(x) + u(x)v'(x). \quad (8.43)$$

我们可以看到其中存在冗余计算： $u(x)$ 和 $v(x)$ 在计算 $f(x)$ 和 $f'(x)$ 时都需要被求值。

如果因子 $u(x)$ 和 $v(x)$ 本身又包含子因子，那么就会出现表达式的嵌套重复，其复杂度会迅速增长。

这一问题被称为表达式膨胀（expression swell）。

重点难点解读：

核心概念辨析：本段的核心在于区分“符号微分”与后文将介绍的“自动微分”。符号微分的本质是对数学表达式进行字符串层面的代数变换，输出的是一个完整的、封闭形式的导数公式；而自动微分是对计算过程（程序执行流）的分解与追踪。理解这一区别是掌握现代深度学习框架底层原理的前提。

表达式膨胀的根源：公式 (8.42) 和 (8.43) 直观展示了问题所在。符号微分严格遵循莱布尼茨乘积法则等解析规则，但不会主动识别并复用中间变量（如 $u(x)$ 、 $v(x)$ 的值）。当网络层数加深或表达式嵌套

时，相同的子表达式会在导数公式中被反复展开和重写，导致公式规模呈指数级爆炸，这使其无法直接应用于深层神经网络。

优势与局限的权衡：符号微分兼具“机器精度”和“避免人为推导错误”两大优势，且克服了数值微分随参数数量线性增长的扩展性问题。但其致命缺陷——表达式膨胀以及对控制流（循环、条件分支等）的不支持，使其在实际工程中被自动微分所取代。这段论述为后续引入自动微分做了关键的理论铺垫。

5 P5 符号微分的表达式膨胀示例

As a further illustration, consider a function that is structured like two layers of a neural network (Grosse, 2018) with a single input x , a hidden unit with activation z , and an output y in which

$$z = h(w_1x + b_1) \quad (8.44)$$

$$y = h(w_2z + b_2) \quad (8.45)$$

where $h(a)$ is the soft ReLU:

$$\zeta(a) = \ln(1 + \exp(a)). \quad (8.46)$$

The overall function is therefore given by

$$y(x) = h(w_2h(w_1x + b_1) + b_2) \quad (8.47)$$

and the derivative of the network output with respect to w_1 , evaluated symbolically, is given by

$$\frac{\partial y}{\partial w_1} = \frac{w_2x \exp(w_1x + b_1 + b_2 + w_2 \ln[1 + e^{w_1x+b_1}])}{(1 + e^{w_1x+b_1})(1 + \exp(b_2 + w_2 \ln[1 + e^{w_1x+b_1}]))}. \quad (8.48)$$

As well as being significantly more complex than the original function, we also see redundant computation where expressions such as $w_1x + b_1$ occur in several places.

翻译：

作为进一步的说明，考虑一个结构类似于两层神经网络的函数（Grosse, 2018），该网络具有单个输入 x 、一个激活值为 z 的隐藏单元以及输出 y ，其定义如下：

$$z = h(w_1x + b_1), \quad (8.44)$$

$$y = h(w_2z + b_2), \quad (8.45)$$

其中 $h(a)$ 为 soft ReLU 函数：

$$\zeta(a) = \ln(1 + \exp(a)). \quad (8.46)$$

因此，整体函数可表示为：

$$y(x) = h(w_2 h(w_1 x + b_1) + b_2). \quad (8.47)$$

通过符号微分求得的网络输出对 w_1 的导数为：

$$\frac{\partial y}{\partial w_1} = \frac{w_2 x \exp(w_1 x + b_1 + b_2 + w_2 \ln[1 + e^{w_1 x + b_1}])}{(1 + e^{w_1 x + b_1})(1 + \exp(b_2 + w_2 \ln[1 + e^{w_1 x + b_1}]))}. \quad (8.48)$$

除了复杂度显著高于原函数外，我们还可以看到明显的冗余计算，例如表达式 $w_1 x + b_1$ 在公式中多次重复出现。

重点难点解读：

公式 (8.48) 的直观冲击：该公式是“表达式膨胀”最直接的视觉证据。原函数 (8.47) 仅有两层嵌套，结构清晰简洁；而其符号导数 (8.48) 却变成了一个极其冗长的分式，包含多层指数与对数嵌套。这生动说明了：即使对于极浅的网络，符号微分生成的导数表达式也会在复杂度上发生质变，对于深层网络而言更是完全不可行。

冗余计算的本质：文中指出 $w_1 x + b_1$ 等子表达式在导数公式中反复出现，这正是符号微分的根本缺陷。符号微分将导数视为一个独立的数学对象进行化简，而无法像程序执行那样将中间结果存储并复用。在实际计算中，这意味着大量重复的浮点运算，严重浪费计算资源。而自动微分（反向模式）正是通过记录计算图并复用前向传播中的中间变量来彻底解决这一问题。

Soft ReLU 的选择意图：作者特意选用 soft ReLU（即 $\ln(1 + e^a)$ ）而非标准 ReLU 作为示例，是因为 soft ReLU 处处光滑可导，其符号导数具有封闭形式，适合用于演示符号微分的机制。若使用标准 ReLU，其导数在非零点为常数、在零点不可导，反而无法充分展现表达式膨胀的问题。这一选择体现了作者在理论阐释上的严谨性。

6 P6 符号微分的局限性：封闭形式与控制流

A further major drawback with symbolic differentiation is that it requires that the expression to be differentiated is expressed in closed form.

It therefore excludes important control flow operations such as loops, recursions, conditional execution, and procedure calls, which are valuable constructs that we might wish to use when defining the network

function.

翻译：

符号微分还有一个主要缺点，即它要求被微分的表达式必须以封闭形式（closed form）给出。

因此，它无法处理重要的控制流操作，例如循环、递归、条件执行以及过程调用，而这些都是定义网络函数时我们可能希望使用的有价值构造。

重点难点解读：

“封闭形式”的含义与限制：所谓封闭形式，是指由有限个基本运算和初等函数组合而成的显式数学表达式。符号微分本质上是对这种静态表达式树进行代数变换，因此它只能作用于预先完全展开的公式。这意味着任何依赖于运行时数据才能确定结构的计算逻辑都无法被符号微分处理，这从根本上限制了其适用范围。

控制流在现代神经网络中的重要性：文中列举的循环、递归、条件分支等并非边缘特性，而是构建现代复杂模型的核心要素。例如，循环神经网络（RNN）依赖时间步上的循环展开，注意力机制中的变长序列处理依赖条件逻辑，神经架构搜索甚至涉及动态子图选择。符号微分对这些构造的排斥，使其完全无法适配当代深度学习的研究范式。

与自动微分的本质对比：这段论述实际上为引出自动微分做了最后的理论铺垫。自动微分之所以能够取代符号微分，关键在于它对**程序执行轨迹**而非**数学表达式**进行微分。只要程序可以运行并产生数值输出，无论其中包含多少控制流，自动微分都能通过追踪实际执行路径来正确计算梯度。这一从“表达式中心”到“执行程序中心”的范式转换，是理解现代深度学习框架设计哲学的关键。

7 P7 自动微分：原理与优势

We therefore turn to the fourth technique for evaluating derivatives in neural networks called *automatic differentiation*, also known as ‘autodiff’ or ‘algorithmic differentiation’ (Baydin et al., 2018).

Unlike symbolic differentiation, the goal of automatic differentiation is not to find a mathematical expression for the derivatives but to have the computer automatically generate the code that implements the gradient calculations given only the code for the forward propagation equations.

It is accurate to machine precision, just as with symbolic differentiation, but is more efficient because

it is able to exploit intermediate variables used in the definition of the forward propagation equations and thereby avoid redundant evaluations.

It is important to note that not only can automatic differentiation handle conventional closed-form mathematical expressions but it can also deal with flow control elements such as branches, loops, recursion, and procedure calls, and is therefore significantly more powerful than symbolic differentiation.

Automatic differentiation is a well-established field with broad applicability that was developed largely outside of the machine learning community.

Modern deep learning is a largely empirical process, involving evaluating and comparing different architectures, and automatic differentiation therefore plays a key role in enabling this experimentation to be done accurately and efficiently.

翻译：

因此，我们转向神经网络中求导的第四种技术，即自动微分（automatic differentiation），也被称为“autodiff”或“算法微分”（Baydin et al., 2018）。

与符号微分不同，自动微分的目标并非寻找导数的数学表达式，而是让计算机仅根据前向传播方程的代码，自动生成实现梯度计算的代码。

它与符号微分一样能够达到机器精度，但效率更高，因为它能够利用前向传播方程定义中使用的中间变量，从而避免冗余计算。

需要注意的是，自动微分不仅能处理传统的封闭形式数学表达式，还能处理分支、循环、递归和过程调用等控制流元素，因此其能力显著强于符号微分。

自动微分是一个成熟且应用广泛的领域，其发展主要是在机器学习社区之外完成的。

现代深度学习在很大程度上是一个经验性过程，涉及对不同架构的评估与比较，因此自动微分在确保这一实验过程准确高效方面发挥着关键作用。

重点难点解读：

核心范式转变：从表达式到代码：理解自动微分的关键在于认识到它操作的对象是**程序而非公式**。符号微分输出的是一个新公式，而自动微分输出的是一段可执行代码（或计算图）。这意味着它将微积分问题转化为了程序变换问题，从而天然支持所有编程语言特性，彻底打破了封闭形式的限制。

中间变量复用与计算效率：文中提到自动微分能“利用中间变量避免冗余计算”，这对应了反向模式自

动微分（reverse-mode autodiff）的核心机制。在前向传播时，所有中间结果被缓存；在反向传播时，梯度计算直接复用这些缓存值。这正是解决前文所述“表达式膨胀”问题的根本方案，使得深层网络的梯度计算复杂度与前向传播同阶。

学科交叉的历史背景：作者特别指出自动微分“主要在机器学习社区之外发展”，这一点值得注意。自动微分源于科学计算、优化理论和编译器研究，已有数十年历史。深度学习对其大规模采用是相对较晚的事。了解这一背景有助于理解为何 PyTorch、JAX 等框架的设计哲学更接近通用编程语言而非传统数学软件，也提醒我们在遇到梯度相关问题时，可从更广泛的数值计算文献中寻找解决方案。

8 P8 自动微分的核心思想与前向模式

The key idea of automatic differentiation is to take the code that evaluates a function, for example the forward propagation equations that evaluate the error function for a neural network, and augment the code with additional variables whose values are accumulated during code execution to obtain the required derivatives.

There are two principal forms of automatic differentiation, known as forward mode and reverse mode.

We start by looking at forward mode, which is conceptually somewhat simpler.

翻译：

自动微分的核心思想是：获取用于计算函数的代码（例如神经网络中用于计算误差函数的前向传播方程），并在执行过程中通过增加额外的变量来累积数值，从而获得所需的导数。

自动微分主要有两种形式，分别称为前向模式（forward mode）和反向模式（reverse mode）。

我们首先来看前向模式，它在概念上相对更为简单。

重点难点解读：

“代码增强”而非“公式推导”：译文中的“augment the code”是理解自动微分实现机制的关键。它表明自动微分不是在数学层面推导导数公式，而是在程序层面修改或包装原始代码。具体而言，系统会在每个原始变量旁附加一个存储导数（或梯度）的伴随变量，并重载基本运算使其同时计算函数值与导数值。这种“对偶数”（dual number）或“磁带记录”机制，正是自动微分区别于符号微分和数值微分的本质特征。

前向与反向模式的计算复杂度差异：虽然文中仅提及前向模式“概念更简单”，但理解两者区别至关重要。前向模式的计算代价与输入维度成正比，适合输入少、输出多的场景；而反向模式的计算代价与输出维度成正比，适合输出少、输入多的场景。由于神经网络的损失函数通常是标量输出、百万级参数输入，反向模式成为深度学习标准选择。作者先讲前向模式，纯粹是出于教学上的由浅入深，而非暗示其更常用。

中间变量的“累积”语义：文中“accumulated during code execution”一词需精确理解。在前向模式中，导数是随着前向计算同步向前累积的（链式法则从左到右）；而在反向模式中，梯度是在前向计算完成后，从输出向输入反向累积的（链式法则从右到左）。这一“累积方向”的差异决定了两种模式的内存访问模式、缓存策略以及是否支持原地操作等工程细节，也是后续理解计算图构建与梯度检查点（gradient checkpointing）等技术的基础。

9 P9 介绍一下深度学习里的自动微分方法

自动微分（Automatic Differentiation，简称 AD）是深度学习框架（如 PyTorch、TensorFlow）的基石技术。它让复杂的神经网络训练变得可能——我们只需定义好网络结构（前向传播），框架就能自动、精确地计算出训练所需的梯度，用于更新模型参数。

它的核心思想很简单：将任何复杂的函数，都分解成一系列简单的、已知求导规则的基本运算（如加减乘除、指数、对数），然后依靠微积分的链式法则（Chain Rule），将这些基本运算的导数一步步组合起来，从而得到整个函数的导数。这就像是把一道复杂的数学题，拆解成无数道小学生都会做的四则运算题，再把结果综合起来。

为了让你更清晰地理解，这里用一个表格总结了它的两种主要工作模式：

实现机制：计算图

自动微分的实现离不开计算图（Computational Graph）。在 PyTorch、MindSpore 等框架中，当我们执行张量运算时，框架会动态构建一个有向无环图（DAG）。

叶子节点：通常是输入数据或模型参数。

中间节点：代表一个基本运算（如加法、矩阵乘法）。

根节点：代表最终的输出（如损失值）。

例如，对于函数 $z = \log(xy)$ ，其计算图会记录下 x 和 y 先经过乘法运算得到中间结果 v ，再经过对数运算得到 z 的过程。

前向传播：沿着计算图从输入到输出，计算出每个节点的值和最终的损失。

表 1: 自动微分前向模式与反向模式对比

特性	前向模式 (Forward Mode)	反向模式 (Reverse Mode)
计算方向	从输入到输出，与前向传播同方向	从输出到输入，先正向计算，再反向传播
工作原理	在计算每个中间变量的同时，计算它对输入的导数	先完整执行前向计算并记录计算图，然后从最终输出开始，反向计算每个中间变量对输出的导数（梯度）
效率特点	一次计算，只能得到一个输入的导数	一次计算，能得到所有输入的导数
内存消耗	通常较低，不需要保存大量中间结果	较高，需要保存前向计算的所有中间结果，用于反向计算
适用场景	输入维度较小，输出维度较大的场景	输入维度远大于输出维度的场景 。深度学习训练正是这种情况（模型参数百万级，损失函数输出是一个标量），因此 反向模式是主流选择

反向传播（反向模式）：从输出节点（根）开始，沿着计算图的反方向，利用链式法则，将最终损失的梯度一层层传回给每个节点。当梯度传播到参数节点时，就得到了我们所需的参数梯度。

为了让效率更高，现代深度学习框架在实现时，通常会存储“向量-雅可比矩阵乘积”（Vector-Jacobian Product, VJP），而不是完整且巨大的雅可比矩阵，以节省内存。

实践中的常见操作

在深度学习框架（如 PyTorch）中，有几个基于自动微分机制的核心操作，它们是我们训练模型时最常打交道的工具：

1. **requires_grad**: 这是一个张量（Tensor）的属性标志。当它被设置为 **True** 时，框架会追踪该张量的所有操作，以构建计算图，从而能够在反向传播时计算其梯度。
 - 在实践中的默认行为是：模型（神经网络）的参数（如权重和偏置）默认开启此标志，因为我们需要更新它们；
 - 而输入数据（如图像、文本）通常不开启此标志，以节省内存和计算资源。
2. **torch.no_grad()**（上下文管理器）：这是一个用于临时禁用梯度计算的上下文管理器。凡是在其缩进范围内的所有操作，都不会被追踪记录到计算图中。
 - 它通常被用于**模型推理（测试/验证）**或执行**参数更新**（如优化器的 **step()** 操作）等不需要梯度的环节；
 - 使用它可以显著降低内存消耗并提升计算速度，是工程实践中的常用优化手段。
3. **.backward()**（方法）：这是触发整个反向传播过程的“开关”方法。

- 当我们在一个张量（通常是损失值 `loss`）上调用 `.backward()` 时，框架会从该张量开始，沿着预先构建好的计算图，反向地应用链式法则进行梯度传播；
- 最终，计算出的梯度值会被累加存储到所有叶子节点（即 `requires_grad=True` 的张量）的 `.grad` 属性中，供优化器使用。

总结来说，自动微分通过将复杂函数分解为基础运算并应用链式法则，以反向模式为核心，为深度学习提供了高效、精确的梯度计算方法，是驱动现代 AI 模型训练的关键引擎。